

Ecommify: Diseño de Base de Datos y Arquitectura Híbrida

Este documento técnico detalla el diseño conceptual y lógico del módulo transaccional para el proyecto Ecommify, integrando capacidades avanzadas de PostgreSQL y una arquitectura híbrida con MongoDB.

1. Resumen Ejecutivo

El sistema Ecommify adopta un enfoque de arquitectura polígloa para maximizar la eficiencia en el manejo de datos transaccionales y analíticos. Se ha seleccionado PostgreSQL como motor principal para el núcleo transaccional debido a su cumplimiento estricto de ACID, y MongoDB para el manejo de catálogos y reseñas con alta variabilidad. El diseño se optimiza para el dataset de Olist (Brazilian E-commerce), utilizando tipos de datos nativos avanzados y extensiones geoespaciales.

2. Diseño Conceptual (Modelo Entidad-Relación)

El modelo se ha normalizado a Tercera Forma Normal (3FN) para garantizar la integridad de los datos y minimizar la redundancia.

- **Customers:** Almacena información única de clientes y su ubicación geográfica.
- **Orders:** Entidad central que gestiona el ciclo de vida de las compras.
- **Order_Items:** Detalle de productos, precios y logística de envío por ítem.
- **Products:** Repositorio de bienes con especificaciones técnicas complejas.
- **Payments:** Registro de transacciones financieras y métodos de pago.

3. Diseño Lógico - Módulo PostgreSQL

Implementación de tipos de datos avanzados para optimizar el almacenamiento y las consultas:

Campo	Tipo de Dato	Justificación
product_specifications	JSONB	Permite esquemas flexibles para atributos variables de productos.

Campo	Tipo de Dato	Justificación
product_photos	ARRAY TEXT[]	Almacena múltiples URLs de imágenes sin requerir tablas adicionales.
promotion_period	TSTZRANGE	Gestión nativa de rangos temporales para promociones y descuentos.

Extensiones y Optimizaciones

- **PostGIS:** Utilizada para el cálculo de distancias geográficas entre vendedores y clientes, permitiendo una estimación precisa del costo de flete.
- **pg_trgm:** Implementada para búsquedas de texto con tolerancia a errores (fuzzy search) en el catálogo de productos.
- **Particionamiento:** La tabla Orders se particiona por rango de fechas (mensual) para facilitar el mantenimiento y mejorar el rendimiento de consultas históricas.

4. Matriz de Decisiones Arquitectónicas (PostgreSQL vs. MongoDB)

Análisis basado en el Teorema CAP y requisitos funcionales.

Módulo / Datos	Tecnología	Criterio CAP	Razón Técnica
Transacciones y Pagos	PostgreSQL	Consistencia (C)	Garantía ACID absoluta para evitar inconsistencias en el flujo de dinero.
Reseñas y Comentarios	MongoDB	Disponibilidad (A)	Escalabilidad horizontal para grandes volúmenes

Módulo / Datos	Tecnología	Criterio CAP	Razón Técnica
			de datos no críticos.
Catálogo de Productos	Híbrido	P y C	PostgreSQL para stock/precio; MongoDB para metadatos enriquecidos.

5. Anexos: Scripts SQL Preliminares

```
-- Creación de tabla particionada
CREATE TABLE orders (
    order_id UUID PRIMARY KEY,
    customer_id UUID,
    status TEXT,
    purchase_timestamp TIMESTAMPTZ,
    updated_at TIMESTAMPTZ DEFAULT NOW()
) PARTITION BY RANGE (purchase_timestamp);

-- Ejemplo de uso de JSONB y Arrays
CREATE TABLE products (
    product_id UUID PRIMARY KEY,
    specifications JSONB,
    photos TEXT[],
    promotion_period TSTZRANGE
);

-- Trigger para actualización automática de updated_at
CREATE OR REPLACE FUNCTION update_timestamp()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```